



Universidad Autónoma de Baja California
Facultad de Ingeniería Mexicali
Ingeniería en Computación

Reporte de Práctica 06
Estructuras de Control y Ciclos en
Ensamblador

Organización y Arquitectura de Computadoras
Ing. Félix Arredondo Santamaría

Moya Monreal Erick Anselmo — 1110604

Fecha de entrega
17/ABRIL/2026

Mexicali, Baja California
30 de mayo de 2026

Índice

1. Introducción	2
2. Problemática y Justificación	2
3. Objetivos de la práctica	2
4. Marco Teórico	3
4.1. Control de Flujo en Lenguaje Ensamblador	3
4.2. La Instrucción LOOP y el Contador ECX	3
4.3. Equivalencia con <code>for</code> , <code>while</code> y <code>do-while</code>	4
4.4. Direccionamiento de Arreglos y el Operador <code>LENGTHOF</code>	5
4.5. Algoritmos de Ordenamiento con Ciclos Anidados	6
5. Procedimiento	6
6. Resultados con pruebas	7
6.1. Ejercicio 1: LOOP básico	7
6.2. Ejercicio 2: Salto condicional dentro de un ciclo	8
6.3. Ejercicio 3: <code>for</code> , <code>while</code> y <code>do-while</code>	9
6.4. Ejercicio 4: Factorial de 10	11
6.5. Ejercicio 5: Resumen del capítulo 4 de Irvine	12
6.6. Ejercicio 6: Recorrido de un arreglo con LOOP	12
6.7. Ejercicio 7: Ordenamiento ascendente	13
7. Conclusiones	15
Referencias	16
A. Repositorio de código	16

1. Introducción

El flujo de ejecución de un programa rara vez avanza en línea recta. La inmensa mayoría de los algoritmos repiten un conjunto de instrucciones un número determinado de veces o mientras se cumpla una condición, y es precisamente esa capacidad de iterar la que permite procesar arreglos, acumular sumas, calcular productos o reorganizar datos. En un lenguaje de alto nivel estas repeticiones se expresan con construcciones como **for**, **while** o **do-while**, que el compilador traduce silenciosamente a saltos sobre el contador de programa.

En lenguaje ensamblador, en cambio, no existen tales palabras reservadas, el programador debe construir cada ciclo a partir de etiquetas, comparaciones y saltos. En esta práctica se exploran los mecanismos de control de flujo de **MASM x86**, empezando por la instrucción dedicada **LOOP**, que combina en una sola operación el decremento de un contador y el salto condicional, para después reproducir manualmente con **CMP** y saltos las tres estructuras de repetición clásicas. Con estas herramientas se resuelven cuatro problemas concretos, el cálculo de un factorial, el doblado iterativo de un valor con salida temprana, el recorrido de un arreglo cuya longitud se obtiene automáticamente y el ordenamiento ascendente de una lista de números.

2. Problemática y Justificación

A diferencia de un lenguaje de alto nivel, donde una sola línea **for(i=0; i<n; i++)** encapsula la inicialización, la condición y el incremento de un ciclo, el ensamblador obliga a descomponer esa misma idea en sus partes elementales, una etiqueta que marca el inicio del cuerpo, una comparación que decide si continuar, y un salto que regresa el control o lo transfiere fuera del ciclo [1].

La instrucción **LOOP** simplifica el caso del ciclo contado porque usa el registro **ECX** como contador implícito, pero se vuelve insuficiente cuando la repetición depende de una condición arbitraria, situación en la que hay que recurrir a **CMP** y a los saltos condicionales (**JG**, **JE**, **JGE**, **JNZ**). Problemas como el ordenamiento exigen ciclos anidados y el intercambio cuidadoso de elementos en memoria, lo que pone a prueba el manejo de punteros y el respeto del contador externo cuando el interno también utiliza **ECX**.

3. Objetivos de la práctica

Objetivo General

Comprender e implementar en lenguaje ensamblador **MASM x86** las estructuras de control iterativas, empleando tanto la instrucción **LOOP** como la construcción manual de

ciclos mediante comparaciones y saltos condicionales, para resolver problemas de cálculo, recorrido de arreglos y ordenamiento.

Objetivos Específicos

- **Comprobar** el funcionamiento de la instrucción `LOOP` con el registro `ECX` como contador implícito en un ciclo de iteraciones fijas.
- **Combinar** un salto condicional dentro de un ciclo para abandonarlo de forma temprana cuando un valor supera una referencia, aun cuando el contador no se haya agotado.
- **Reproducir** con `CMP` y saltos las estructuras de control `for`, `while` y `do-while` de un lenguaje de alto nivel.
- **Aplicar** la teoría de ciclos al cálculo del factorial de 10 mediante multiplicaciones sucesivas.
- **Recorrer** un arreglo asignando cada elemento al registro `EAX`, obteniendo su longitud automáticamente con el operador `LENGTHOF` en lugar de fijarla a mano.
- **Implementar** el ordenamiento ascendente de un arreglo de enteros utilizando ciclos anidados y el intercambio de elementos en memoria.

4. Marco Teórico

4.1. Control de Flujo en Lenguaje Ensamblador

El procesador ejecuta instrucciones de manera secuencial, avanzando el registro contador de programa (EIP) tras cada operación. Para alterar ese orden lineal y construir repeticiones o decisiones se emplean las instrucciones de salto, que modifican EIP para que apunte a una etiqueta distinta de la siguiente instrucción [1]. Un salto puede ser **incondicional** (`JMP`), que transfiere el control sin condición alguna, o **condicional** (`JE`, `JG`, `JGE`, `JLE`, `JNZ`, entre otros), que solo se efectúa si las banderas del registro de estado, fijadas normalmente por una instrucción `CMP` previa, cumplen cierta condición. Con estos dos ingredientes, etiquetas y saltos, se puede reconstruir cualquier estructura de control de un lenguaje de alto nivel.

4.2. La Instrucción `LOOP` y el Contador `ECX`

`LOOP` es una instrucción especializada en ciclos contados. Cada vez que se ejecuta realiza dos acciones de manera atómica: decrementa el registro `ECX` en una unidad y, si el resultado es distinto de cero, salta a la etiqueta indicada; cuando `ECX` llega a cero el

salto no se realiza y la ejecución continúa con la instrucción siguiente [2]. De este modo, basta con cargar en **ECX** el número de repeticiones deseado antes de entrar al ciclo. Esta comodidad tiene una consecuencia, **ECX** queda reservado como contador, por lo que si el cuerpo del ciclo necesita usarlo para otra cosa, su valor debe preservarse en la pila con **PUSH** y **POP**, situación que se presenta de manera natural en los ciclos anidados. La Figura 1 ilustra el mecanismo interno de **LOOP**.

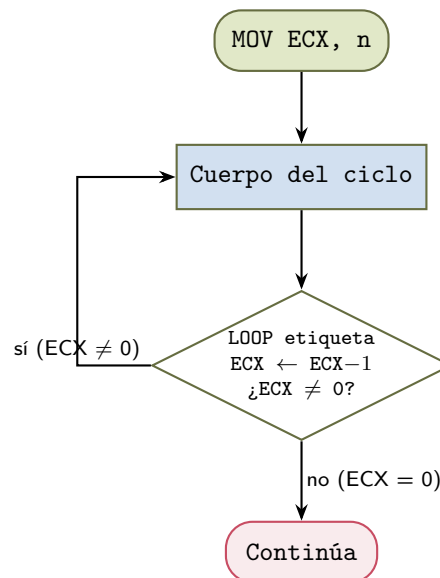
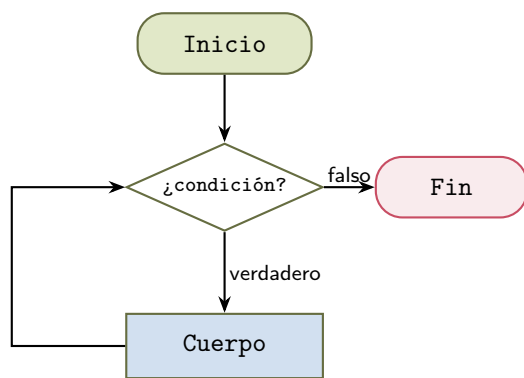


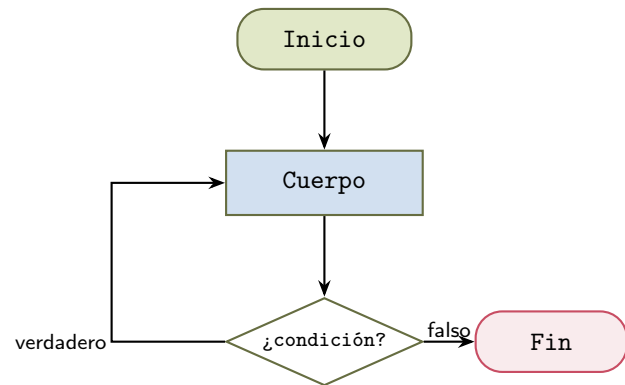
Figura 1: Mecanismo de la instrucción **LOOP**: decreenta **ECX** y, mientras no sea cero, retorna al cuerpo del ciclo.

4.3. Equivalencia con **for**, **while** y **do-while**

Las tres estructuras de repetición de los lenguajes de alto nivel se distinguen por el momento en que evalúan su condición. El ciclo **for** y el **while** la comprueban **antes** de ejecutar el cuerpo (ciclos preprobados), de modo que pueden no ejecutarse ni una sola vez; en ensamblador se traducen colocando un **CMP** seguido de un salto condicional al inicio, y un **JMP** al final que regresa a esa comprobación [3]. El **do-while**, en cambio, evalúa la condición **al final** (ciclo posprobado), garantizando al menos una ejecución del cuerpo; su traducción coloca la comparación y el salto condicional al final del bloque. Entender esta diferencia es clave para elegir la plantilla correcta al codificar un algoritmo. La Figura 2 contrasta ambos esquemas.



(a) Preprobado (**for** / **while**): puede no ejecutar el cuerpo.



(b) Posprobado (**do-while**): ejecuta el cuerpo al menos una vez.

Figura 2: Diferencia estructural entre los ciclos preprobados y el posprobado según el momento en que evalúan la condición.

4.4. Direccionamiento de Arreglos y el Operador `LENGTHOF`

Un arreglo se almacena como una secuencia contigua de elementos en memoria. Para recorrerlo se utiliza el direccionamiento indexado, en el que la dirección de un elemento se calcula sumando a la dirección base (`OFFSET` del arreglo) un desplazamiento proporcional al índice [4]. Un aspecto crítico es no codificar la longitud del arreglo de forma manual, pues cualquier cambio en la declaración la dejaría desactualizada. El operador `LENGTHOF` resuelve este problema calculando en tiempo de ensamblado el número de elementos de un arreglo a partir de su declaración, de manera que el ciclo siempre itera el número exacto de veces sin importar cuántos elementos se agreguen o quiten. La Figura 3 muestra la disposición en memoria de la cadena "HOLA" y el cálculo de su dirección.

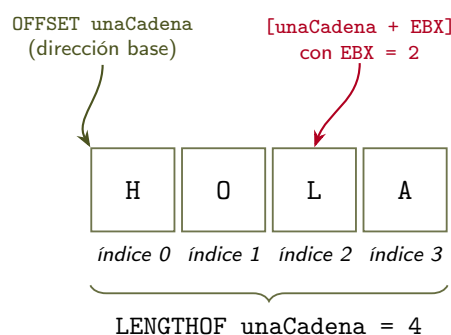


Figura 3: Disposición en memoria de la cadena "HOLA": la dirección de cada byte se obtiene sumando el índice a la base, y `LENGTHOF` entrega el número de elementos.

4.5. Algoritmos de Ordenamiento con Ciclos Anidados

Ordenar un arreglo es un problema clásico que requiere comparar y reorganizar sus elementos. El método de la burbuja (*bubble sort*) recorre repetidamente la lista comparando pares de elementos adyacentes e intercambiándolos cuando están en el orden incorrecto, de forma que en cada pasada el mayor elemento “flota” hacia el final [3]. Su implementación en ensamblador necesita dos ciclos anidados, uno externo que controla el número de pasadas y uno interno que realiza las comparaciones e intercambios. Como ambos usarían el contador ECX, el del ciclo externo se preserva en la pila mientras el interno trabaja, ilustrando la importancia de gestionar correctamente los registros compartidos. La Figura 4 muestra una comparación e intercambio del primer par adyacente.

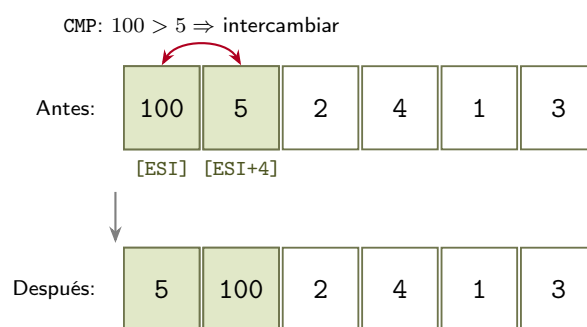


Figura 4: Una comparación del método de la burbuja: como $[ESI] > [ESI+4]$, los dos elementos adyacentes se intercambian en memoria.

5. Procedimiento

La práctica se organizó como una serie de ejercicios, cada uno enfocado en un mecanismo de control de flujo distinto y apoyándose en lo aprendido en el anterior.

1. **Ejercicio 1 — LOOP básico.** Se construyó un ciclo de cinco iteraciones con LOOP que incrementa un registro en cada vuelta, comprobando el funcionamiento elemental del contador ECX.
2. **Ejercicio 2 — Salto condicional dentro de un ciclo.** Dentro de un ciclo de cinco iteraciones se dobla el valor de un registro en cada vuelta y se sale anticipadamente con un salto condicional cuando supera un valor de referencia.
3. **Ejercicio 3 — for, while y do-while.** Se reprodujeron con CMP y saltos las tres estructuras de repetición clásicas de un lenguaje de alto nivel.
4. **Ejercicio 4 — Factorial de 10.** Se calculó el factorial de 10 mediante un ciclo de multiplicaciones sucesivas controlado por LOOP.

5. **Ejercicio 5 — Resumen del capítulo 4.** Se elaboró un resumen del capítulo 4 del libro *Assembly Language for x86 Processors* de Kip Irvine.
6. **Ejercicio 6 — Recorrido de un arreglo con LOOP.** Se recorrió una cadena asignando cada carácter a EAX, obteniendo el número de iteraciones con LENGTHOF.
7. **Ejercicio 7 — Ordenamiento ascendente.** Se ordenó un arreglo de enteros de menor a mayor mediante el método de la burbuja con ciclos anidados.

6. Resultados con pruebas

6.1. Ejercicio 1: LOOP básico

El primer ejercicio comprueba el funcionamiento elemental de la instrucción LOOP. Se carga en ECX el número de iteraciones (5) y en EAX un acumulador en cero; en cada vuelta del ciclo se incrementa EAX y LOOP se encarga de decrementar ECX y repetir hasta agotarlo. El Código 1 muestra la implementación.

```

1      MOV ECX, 5           ; numero de iteraciones
2      MOV EAX, 0          ; acumulador
3
4      SALT01:
5          INC EAX          ; EAX = EAX + 1
6          LOOP SALT01      ; ECX-- ; si ECX != 0 -> SALT01

```

Listing 1: Ciclo de cinco iteraciones con LOOP que incrementa un registro.

Análisis de instrucción

Instrucción

```

1      INC EAX
2      LOOP SALT01

```

Descripción Incrementa el acumulador y delega en LOOP el control del ciclo: decrementa ECX y salta mientras no llegue a cero.

El uso de LOOP radica en que condensa en una sola instrucción lo que de otro modo requeriría un DEC ECX seguido de un JNZ. Al iniciar con ECX = 5, el cuerpo se ejecuta cinco veces y EAX termina valiendo 00000005, mientras que ECX queda en 00000000.

Registers	
EAX = 00000005	EBX = 0071D000
ECX = 00000000	EDX = 00411005
ESI = 00411005	EDI = 00411005
EIP = 0041101D	ESP = 008FFE74
EBP = 008FFE80	EFL = 00000206

Figura 5: Estado de los registros tras el ciclo LOOP: EAX alcanza 00000005 (5) y ECX queda en 00000000.

6.2. Ejercicio 2: Salto condicional dentro de un ciclo

El segundo ejercicio combina un ciclo contado con un salto condicional que permite abandonarlo antes de tiempo. Dentro de un ciclo de cinco iteraciones, el registro EAX (inicializado en 2) se duplica en cada vuelta sumándose a sí mismo; en cuanto su valor supera la referencia de 32, un JG rompe el ciclo. El Código 2 muestra la rutina.

```

1      MOV ECX, 5                ; ciclo "largo" de respaldo
2      MOV EAX, 2                ; valor inicial
3
4      SALT02:
5          ADD EAX, EAX           ; EAX = EAX * 2
6          CMP EAX, 32           ; supero la referencia?
7          JG SALT03             ; si EAX > 32 -> salir
8          LOOP SALT02
9
10     SALT03:
11     NOP                       ; destino de la salida temprana

```

Listing 2: Doblado de un valor con salida temprana al superar la referencia.

Análisis de instrucción

Instrucción

```

1      ADD EAX, EAX
2      CMP EAX, 32
3      JG SALT03

```

Descripción Duplica el valor de EAX, lo compara contra la referencia y, si la excede, salta fuera del ciclo sin esperar a que ECX se agote.

ADD EAX, EAX es una forma eficiente de multiplicar por dos, ya que equivale a desplazar el valor un bit a la izquierda. Partiendo de 2, la secuencia es 4, 8, 16, 32 y 64; en la quinta vuelta EAX vale 00000040 (64 en decimal), que es mayor que 32, por lo que JG (*Jump*

if Greater) transfiere el control a **SALT03**. El ciclo se dimensionó con cinco iteraciones precisamente para garantizar que la condición de referencia se alcance antes de que **LOOP** lo termine por contador, demostrando que la salida fue provocada por el salto condicional y no por el agotamiento de **ECX**.

Registers	
EAX = 00000040	EBX = 0071C000
ECX = 00000001	EDX = 00411005
ESI = 00411005	EDI = 00411005
EIP = 0041102C	ESP = 008FFE40
EBP = 008FFE4C	EFL = 00000202

Figura 6: El ciclo se abandona cuando **EAX** supera 32, terminando con el valor 00000040 (64).

6.3. Ejercicio 3: for, while y do-while

El tercer ejercicio reproduce manualmente las tres estructuras de repetición de un lenguaje de alto nivel, evidenciando cómo cada una se distingue por el momento en que evalúa su condición. El Código 3 reúne las tres construcciones.

```

1  ; --- Ciclo FOR: for(i=0; i<10; i++) ---
2      MOV contadorFOR, 0
3  CICLO_FOR:
4      CMP contadorFOR, 10      ; condicion AL INICIO
5      JGE FIN_FOR
6      NOP                      ; cuerpo del ciclo
7      INC contadorFOR          ; incremento
8      JMP CICLO_FOR
9  FIN_FOR:
10     MOV EAX, contadorFOR
11
12  ; --- Ciclo WHILE: while(contador > 0) ---
13     MOV ECX, contadorWHILE
14     MOV EAX, 5
15  CICLO_WHILE:
16     CMP ECX, 0                ; condicion AL INICIO
17     JE FIN_WHILE
18     DEC ECX
19     ADD EAX, ECX
20     JMP CICLO_WHILE
21  FIN_WHILE:
22
23  ; --- Ciclo DO-WHILE: do{...}while(contador > 0) ---

```

```
24     MOV ECX, contadorDOWHILE
25     MOV EDX, 0
26 CICLO_DOWHILE:
27     INC EDX                ; cuerpo: se ejecuta al menos una vez
28     DEC ECX
29     CMP ECX, 0            ; condicion AL FINAL
30     JG CICLO_DOWHILE
```

Listing 3: Equivalentes en ensamblador de for, while y do-while.

Análisis de instrucción

Instrucción

```
1 CICLO_FOR:
2     CMP contadorFOR, 10
3     JGE FIN_FOR
```

Descripción Comprueba la condición de continuación **antes** de ejecutar el cuerpo, característica de los ciclos **for** y **while** (preprobados).

En el **for** y el **while** la comparación encabeza el bloque, si la condición no se cumple desde el principio, el cuerpo no llega a ejecutarse. La diferencia conceptual entre ambos es que el **for** agrupa la inicialización, la prueba y el incremento en un patrón fijo, mientras que el **while** solo conserva la prueba. El salto **JGE** (*Jump if Greater or Equal*) abandona el ciclo cuando el contador alcanza el límite.

Análisis de instrucción

Instrucción

```
1 CICLO_DOWHILE:
2     INC EDX
3     DEC ECX
4     CMP ECX, 0
5     JG CICLO_DOWHILE
```

Descripción Ejecuta primero el cuerpo y evalúa la condición **al final**, garantizando al menos una iteración (**do-while** posprobado).

El **do-while** invierte el orden: el cuerpo (**INC EDX**) se ejecuta antes de que **CMP** y **JG** decidan si repetir. Por ello, aun cuando el contador llegara ya bajo, el cuerpo correría al menos una vez. Esta distinción entre evaluar la condición al inicio o al final es exactamente la que separa a las tres estructuras en cualquier lenguaje, solo que aquí queda explícita en la posición de las instrucciones de comparación y salto.

6.4. Ejercicio 4: Factorial de 10

El cuarto ejercicio aplica la teoría de ciclos al cálculo de $10! = 3\,628\,800$ mediante multiplicaciones sucesivas. Se inicializa EAX en 1 y ECX en 10; en cada vuelta se multiplica el acumulador por el contador y LOOP lo decrementa, de modo que se computa $10 \times 9 \times 8 \times \dots \times 1$. El Código 4 presenta la rutina.

```
1  MOV EAX, 1           ; acumulador del producto
2  MOV ECX, 10          ; multiplicador inicial = 10
3
4  FACTORIAL_10:
5      MUL ECX           ; EAX = EAX * ECX
6      LOOP FACTORIAL_10 ; ECX-- ; repetir mientras ECX != 0
```

Listing 4: Cálculo del factorial de 10 con multiplicaciones sucesivas.

Análisis de instrucción

Instrucción

```
1  MUL ECX
2  LOOP FACTORIAL_10
```

Descripción Multiplica el acumulador por el valor actual del contador y deja que LOOP lo reduzca, generando el producto $10 \times 9 \times \dots \times 1$.

Aquí se aprovecha una particularidad afortunada: el contador ECX es a la vez el multiplicando que el factorial necesita en cada paso. La instrucción `MUL ECX` multiplica EAX por ECX dejando el resultado en EAX (la parte alta, en este caso nula, iría a EDX). Como `LOOP` hace que ECX tome los valores 10, 9, 8, ..., 1, el acumulador recorre todos los factores y termina con $10! = 3\,628\,800$, que en la ventana de registros se lee como `00375F00` y cabe en los 32 bits de EAX.

Registers	
EAX = 00375F00	EBX = 0071C000
ECX = 00000000	EDX = 00000000
ESI = 00411005	EDI = 00411005
EIP = 00411087	ESP = 008FFE40
EBP = 008FFE4C	EFL = 00000246

Figura 7: Resultado del factorial de 10 acumulado en EAX: `00375F00` (3 628 800).

6.5. Ejercicio 5: Resumen del capítulo 4 de Irvine

El capítulo 4 del libro *Assembly Language for x86 Processors* de Kip Irvine, titulado *Data Transfers, Addressing, and Arithmetic* [1], inicia con las **instrucciones de transferencia de datos**. Presenta MOV como la operación fundamental para copiar datos entre registros y memoria, recalcando sus restricciones, ambos operandos deben ser del mismo tamaño y no es posible mover directamente de memoria a memoria. Introduce además MOVZX y MOVSX para copiar un operando pequeño a uno mayor extendiendo con ceros o con el signo respectivamente, y XCHG para intercambiar el contenido de dos operandos, una operación directamente útil en algoritmos de ordenamiento.

También aborda la **aritmética de enteros básica** con INC, DEC, ADD, SUB y NEG, detallando cómo cada una afecta a las banderas de estado (*Zero, Carry, Sign, Overflow*). La comprensión de estas banderas es esencial, pues son las que los saltos condicionales evalúan para tomar decisiones, el mecanismo de los ciclos.

La tercera sección cubre los **operadores y directivas relacionados con datos**: OFFSET, que devuelve la dirección de una variable; PTR, que fuerza un tamaño de operando; TYPE, LENGTHOF y SIZEOF, que informan respectivamente del tamaño de cada elemento, del número de elementos y del tamaño total de un arreglo.

Después, continúa con el **direccionamiento indirecto**, donde un registro contiene la dirección del dato. Explica los operandos indirectos ([ESI]), los indexados ([ESI + ECX]) y su uso natural para recorrer arreglos. Por último, la sección de **instrucciones JMP y LOOP** formaliza la construcción de ciclos, describe JMP como salto incondicional y LOOP como la instrucción que decrementa ECX y salta mientras no sea cero. El capítulo proporciona el mínimo (mover, operar, direccionar y saltar) con el que se programa cualquier algoritmo en MASMx86.

6.6. Ejercicio 6: Recorrido de un arreglo con LOOP

El sexto ejercicio recorre una cadena de caracteres asignando cada elemento al registro EAX, con la condición de que el número de iteraciones no se fije manualmente sino que se derive de la propia declaración del arreglo. El Código 5 muestra la solución.

```
1 ; .data -> unaCadena BYTE "HOLA"
2
3 MOV     ECX, LENGTHOF unaCadena    ; longitud automatica
4 MOV     EBX, 0                     ; indice = 0
5
6 CICLO_ARREGLO:
7     MOVZX EAX, BYTE PTR [unaCadena + EBX]
8     INC     EBX                     ; siguiente posicion
9     LOOP   CICLO_ARREGLO
```

Listing 5: Recorrido de una cadena cargando cada carácter en EAX.

Análisis de instrucción

Instrucción

```
1 MOV ECX, LENGTHOF unaCadena
```

Descripción Obtiene el número de elementos del arreglo, evitando codificar su longitud de forma manual.

El operador `LENGTHOF` consulta la declaración `unaCadena BYTE "HOLA"` y devuelve 4, el número de bytes que componen la cadena. La ventaja es que si la cadena cambia de tamaño, el ciclo se ajusta automáticamente sin necesidad de tocar el código del bucle, eliminando una fuente común de errores.

Análisis de instrucción

Instrucción

```
1 MOVZX EAX, BYTE PTR [unaCadena + EBX]
2 INC EBX
```

Descripción Carga el byte ubicado en la dirección base más el índice, extendiéndolo con ceros a 32 bits, y avanza el índice al siguiente carácter.

La dirección de cada carácter se calcula con direccionamiento indexado, sumando el índice `EBX` a la dirección base `unaCadena`. El operador `BYTE PTR` aclara que el acceso es de un solo byte, y `MOVZX` (*Move with Zero-Extend*) lo coloca en `EAX` rellenando con ceros los 24 bits superiores, ya que el carácter ocupa 8 bits y el registro 32. En cada vuelta `EBX` avanza una posición y `LOOP` decrementa `ECX`, recorriendo así 'H', 'O', 'L', 'A' en orden. En la última lectura `EAX` contiene `00000041`, el código ASCII de 'A', mientras `EBX` marca el índice `00000004`.

Registers	
EAX = 00000041	EBX = 00000004
ECX = 00000001	EDX = 00000000
ESI = 00411005	EDI = 00411005
EIP = 00411099	ESP = 008FFE40
EBP = 008FFE4C	EFL = 00000202

Figura 8: Recorrido de la cadena: en la última iteración `EAX` contiene `00000041`, el código ASCII de 'A'.

6.7. Ejercicio 7: Ordenamiento ascendente

El último ejercicio ordena de menor a mayor un arreglo de enteros aplicando el método de la burbuja con dos ciclos anidados. El ciclo externo controla el número de pasadas y

el interno compara pares adyacentes, intercambiándolos cuando están desordenados. El Código 6 presenta la implementación.

```

1 ; .data -> unArreglo DWORD 100, 5, 2, 4, 1, 3
2
3     MOV     ECX, LENGTHOF unArreglo
4     DEC     ECX                     ; n-1 pasadas
5
6 CONTROL_EXTERNO:
7     PUSH    ECX                     ; preservar contador externo
8     MOV     ESI, OFFSET unArreglo ; puntero al inicio
9     MOV     EDX, ECX               ; comparaciones de esta pasada
10
11 CONTROL_INTERNO:
12     MOV     EAX, [ESI]             ; elemento actual
13     MOV     EBX, [ESI+4]           ; elemento siguiente
14     CMP     EAX, EBX
15     JLE     NO_CAMBIAR             ; en orden -> no intercambiar
16     MOV     [ESI], EBX             ; intercambio
17     MOV     [ESI+4], EAX
18
19 NO_CAMBIAR:
20     ADD     ESI, 4                 ; avanzar al siguiente par
21     DEC     EDX
22     JNZ     CONTROL_INTERNO
23     POP     ECX                   ; restaurar contador externo
24     LOOP    CONTROL_EXTERNO

```

Listing 6: Ordenamiento ascendente por el método de la burbuja con ciclos anidados.

Análisis de instrucción

Instrucción

```

1     PUSH    ECX
2     ...
3     POP     ECX
4     LOOP    CONTROL_EXTERNO

```

Descripción Preserva el contador del ciclo externo en la pila mientras el ciclo interno reutiliza los registros, restaurándolo antes de la siguiente pasada.

El reto de los ciclos anidados es que ambos querrían usar ECX. La solución es guardar el contador externo con PUSH ECX al inicio de cada pasada y recuperarlo con POP ECX justo antes de LOOP, de modo que el ciclo interno (controlado aquí por EDX) pueda operar libremente sin destruir la cuenta de pasadas. Sin esta técnica, se corrompería el ciclo externo y el ordenamiento nunca terminaría correctamente.

Análisis de instrucción

Instrucción

```
1  CMP EAX, EBX
2  JLE NO_CAMBIAR
3  MOV [ESI], EBX
4  MOV [ESI+4], EAX
```

Descripción Compara dos elementos adyacentes y, si están en orden incorrecto, los intercambia escribiéndolos cruzados en memoria.

El corazón del algoritmo es la comparación de cada par adyacente. Se cargan el elemento actual ([ESI]) y el siguiente ([ESI+4], cuatro bytes más adelante por tratarse de un DWORD); si el actual es menor o igual al siguiente (JLE), ya están en orden y no se hace nada. En caso contrario se intercambian, escribiendo cada valor en la posición del otro. Tras procesar el par, ESI avanza cuatro bytes al siguiente. Repitiendo este barrido, el arreglo 100, 5, 2, 4, 1, 3 queda finalmente ordenado como 1, 2, 3, 4, 5, 100.

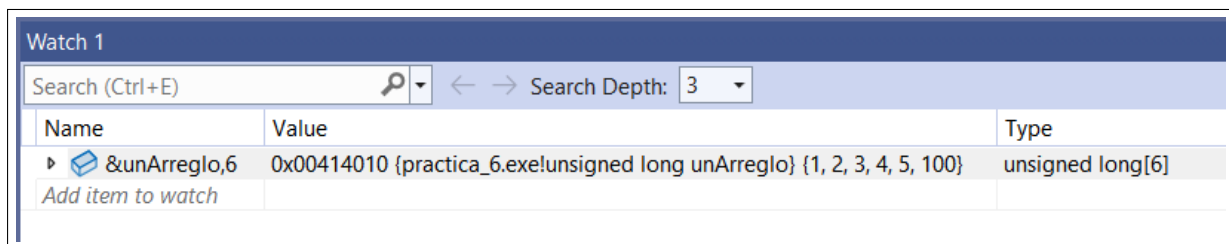


Figura 9: Arreglo ordenado de forma ascendente tras aplicar el método de la burbuja.

7. Conclusiones

Al hacer esta práctica, se comprendió mejor que las estructuras de control que en un lenguaje de alto nivel se dan por sentadas son, a nivel de máquina, combinaciones de comparaciones y saltos sobre el contador de programa.

La instrucción LOOP demostró ser una herramienta cómoda para los ciclos contados al fusionar el decremento y el salto, pero también depende de ECX, que obliga a preservar ese registro en la pila cuando se anidan ciclos, como ocurrió en el ordenamiento.

El uso de LENGTHOF para obtener la longitud del arreglo en lugar de escribirla a mano resultó útil, delegar al ensamblador un dato que puede cambiar evita errores difíciles de rastrear. Implementar el método de la burbuja con direccionamiento indexado y el intercambio de elementos en memoria demuestra cómo se construyen algoritmos completos y complejos en ensamblador.

Referencias

- [1] K. R. Irvine, *Assembly Language for x86 Processors*, 8th. Hoboken, NJ: Pearson, 2020, ISBN: 978-0-13-538165-6.
- [2] K. R. Irvine, *Assembly Language for x86 Processors*, 6th. Upper Saddle River, NJ: Prentice Hall, 2011.
- [3] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM. Morgan Kaufmann, 2016.
- [4] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th. Hoboken, NJ: Pearson Education, Inc., 2019, ISBN: 978-0-13-499719-3.
- [5] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6th. Pearson, 2013.
- [6] A. Elahi, *Computer Systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language*, 2nd. Cham, Switzerland: Springer Nature Switzerland AG, 2022, ISBN: 978-3-030-93448-4. DOI: [10.1007/978-3-030-93449-1](https://doi.org/10.1007/978-3-030-93449-1).
- [7] J. L. Peterson, *Computer Organization and Assembly Language Programming*, Digital Edition. James L. Peterson, 2018.
- [8] Microsoft Corporation, *Windows Console API: WriteConsole, ReadConsole and GetStdHandle Functions*, <https://learn.microsoft.com/en-us/windows/console/console-functions>, Microsoft Learn, 2023.

A. Repositorio de código

El código completo de los ejercicios en **MASM x86** desarrollados durante esta práctica se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ApoloEM/OyAdeC/tree/main/practica6>